atlas.

TECHNISCHES WHITEPAPER · 01

Capability-driven Automation

Von der Business Architecture über semantische Worker-Verträge bis zur kontrollierten und nachvollziehbaren Ausführung.

Der gemeinsame technische Nenner von Wertschöpfung, BPMN, Workern, Identität und Governance.

VISION / ZIELARCHITEKTUR / UMSETZUNGSPLAN

Technische Ausgabe · Version 1.0

24. Oktober 2026

Inhalt

Vom Architekturkonzept über technische Verträge und autorisierte Job-Ausführung bis zum ersten realisierbaren Entwicklungs-Slice.

Executive Summary ↗
1. Problemstellung und strategisches Ziel ↗
2. Architekturprinzipien ↗
3. Referenzarchitektur: Sieben zusammenhängende Ebenen ↗
4. Der Technical Capability Contract ↗
5. Typisierung, Kompatibilität und Versionierung ↗
6. Registrierung, Graph und Panorama ↗
7. Identität und Policy-basierte Autorisierung ↗
8. Runtime-Ausführung: vom expliziten Worker zur Capability-Anforderung ↗
9. Event-Sourcing, Recovery und Datenminimierung ↗
10. APIs und Datenmodell – technischer Schnittstellenentwurf ↗
11. Governance, Bedrohungen und betriebliche Grenzen ↗
12. Roadmap: realistische Umsetzung in vertikalen Slices ↗
13. Empfohlener Proof of Concept ↗
14. Offene Entscheidungen vor der Implementierung ↗
15. Fazit und technische Vision ↗
Referenzen und Ist-Stand (Repository geprüft am 24.10.2026) ↗

Leseschlüssel: Bestehend, teilweise implementiert, vorgeschlagen und empfohlen sind im Text bewusst getrennt. Beispiel-APIs und Capability-Schemas sind als Entwurf markiert.

> **Leitgedanke:** atlas soll lückenlos und überprüfbar zeigen, welcher Geschäftswert durch welche Fähigkeit, welchen Prozess und welche technische Operation entsteht – und mit welcher Identität, unter welcher Policy und über welchen Worker diese Operation tatsächlich ausgeführt wird.

Executive Summary

Unternehmensarchitektur und Prozessautomatisierung werden heute häufig mit unterschiedlichen Werkzeugen beschrieben. Das Architekturmodell erklärt den Geschäftswert und die Abhängigkeiten; der Prozess orchestriert Arbeit; die Integration führt sie aus; das Berechtigungssystem entscheidet über den Zugriff. Die Verbindungen zwischen diesen Ebenen sind oft lediglich implizit.

Die Vision von **Capability-driven Automation** ist ein gemeinsames, semantisch belastbares Modell dieser Beziehungen: **Value Stream → Business Capability → BPMN-Prozess → Technical Capability → Worker Type → konfigurierter Worker → autorisierte Ausführung**. Hinzu kommen Geschäftskennzahlen, operative Ereignisse, technische Abhängigkeiten und nachvollziehbare Entscheidungsnachweise.

Der entscheidende Baustein ist ein **Technical Capability Contract**: ein maschinenlesbarer, versionierter Vertrag, der beschreibt, *welche* fachlich relevante technische Operation ein Worker Type anbietet, *welche Semantik* diese Operation hat, *welche Eingaben und Ergebnisse* gelten, *welche Fehler* auftreten können und *welche Garantien* bezüglich Konsistenz, Wiederholung und Nebenwirkungen tatsächlich bestehen.

Die Architektur ist keine Forderung, die vorhandenen Entra-, LDAP- oder Google-Contacts-Worker zu einem einzigen Universal-Worker zu verschmelzen. Im Gegenteil: Sie macht Gleichheiten **und Unterschiede explizit**. Eine technisch ähnlich benannte Operation ist nicht automatisch austauschbar. Ein Google-Kontakt ist kein Login-Konto, und das Anlegen eines LDAP-Eintrags entspricht nicht zwingend dem Provisionieren einer Entra-Identität.

Die wesentlichen Grundlagen in atlas existieren bereits: Business Capabilities und Value Streams, Bindings in Panorama, BPMN-Engine und langlebige Jobs, Worker Type / Worker / Worker Instance sowie erste identitätsbewusste, policygesteuerte Ausführungswege. **Noch nicht umgesetzt** sind ein allgemein gültiger Technical Capability Contract, der einheitliche mehrschichtige Graph nach ADR-0420 und ein Capability-basiertes, dynamisches Runtime-Routing. Diese Grenzen sind ausdrücklich Teil dieses Papiers.

1. Problemstellung und strategisches Ziel

Ein Unternehmen kann heute modellieren, dass ein *Mitarbeitereintritt* die Capability *Digitale Identität bereitstellen* benötigt. Der ausführbare BPMN-Prozess verwendet anschliessend beispielsweise `entra.create-user`, `ldap.add` oder weitere konkrete Worker-Operationen. Zwischen der geschäftlichen Aussage und der konkreten Implementation fehlen jedoch häufig formale Beziehungen, überprüfbare Verträge und Sichtbarkeit über die ganze Kette.

Das verursacht vier Schwierigkeiten:

1. **Impact-Analyse bleibt unvollständig.** Fällt eine technische Integration aus, ist nicht automatisch ersichtlich, welche Prozesse, Capabilities und Value-Stream-Stufen dadurch beeinträchtigt werden.
2. **Implementierungen sind zu stark an Anbieter gebunden.** Ein Prozess kennt eine Operation, aber nicht unbedingt ihre herstellerunabhängige Absicht oder ihren Vertrag.

3. **Autorisierung und Verantwortung sind schwer durchgängig zu erklären.** Die fachliche Befugnis, einen Prozess auszulösen, ist nicht identisch mit der technischen Berechtigung, im Zielsystem eine Aktion auszuführen.
4. **Architektur ist nicht ausreichend mit tatsächlicher Ausführung verbunden.** Eine Zeichnung kann geplante Abhängigkeiten zeigen; für Betrieb und Revision braucht es zusätzlich beobachtete, dauerhaft belegbare Fakten.

Zielbild: atlas wird zu einer Plattform, in der Facharchitektur, Ausführungsarchitektur und Governance mehrere Sichten auf dieselben eindeutig identifizierten Beziehungen sind. Die Engine bleibt eine BPMN-Engine; Panorama bleibt eine Architektursicht; der Worker-Katalog bleibt das Verzeichnis technischer Implementierungen. Ein gemeinsames semantisches Modell verbindet sie, ohne ihre Verantwortlichkeiten zu vermischen.

2. Architekturprinzipien

Prinzip	Verbindliche Konsequenz
Business Capability ist technologieunabhängig	Geschäftsverantwortung, Inputs/Outputs, KPIs und SLAs gehören ins vorhandene Capability-Register, nicht in Worker-Metadaten.
Technical Capability beschreibt einen Vertrag	Eine Operation wird anhand ihrer beobachtbaren Semantik beschrieben, nicht allein nach ihrem Namen.
Worker Type ≠ Worker ≠ Worker Instance	Typ beschreibt Implementierung; Worker bindet Zielsystem und Identität; Instance führt Jobs aus.
Design Time ≠ Runtime	Katalog, Beziehungen und graphische Sichten dürfen die deterministische Engine nicht mit Laufzeitinterpretation belasten.
Authentisierung ≠ Autorisierung	Eine verknüpfte Identität ist kein Freipass; Policies prüfen Operation, Ressource, Subjekt und Kontext.
Fakten ≠ Modellannahmen	Deklarierte, abgeleitete und beobachtete Beziehungen bleiben getrennt und werden nicht stillschweigend zu einer „Wahrheit“ verschmolzen.
Recovery und Sicherheit zuerst	Routing, Policy-Entscheidungen und externe Side Effects respektieren WAL, `fsync`, Single Writer und Replay.
Keine Scheinaustauschbarkeit	Provider-Unterschiede, fehlende Zusicherungen und inkompatible Operationen bleiben sichtbar.

Diese Prinzipien sind mit den bestehenden Architekturentscheidungen vereinbar, sofern die Umsetzung schrittweise und **nicht** als verdeckter Umbau des Processor-Hot-Paths erfolgt.

3. Referenzarchitektur: Sieben zusammenhängende Ebenen

Ebene 1 – Value Streams. Sie beschreiben die Wertschöpfung aus Sicht der Organisation oder ihrer Kunden. Beispiel: *Eintritt → Arbeitsplatzbereitstellung → produktives Arbeiten → Austritt*.

Ebene 2 – Business Capabilities. Sie beschreiben dauerhaft benötigte Fähigkeiten wie *Identitäten verwalten* oder *Zugänge bereitstellen*, unabhängig davon, welches Produkt sie heute realisiert. Das Register in `api/capability/` speichert unter anderem Scope, Interfaces, Owner, Realizations, `requires`, KPIs und SLAs. Es handelt sich bewusst um eine flache, getaggte Menge ohne Capability-Hierarchie.

Ebene 3 – Prozesse und Entscheidungen. BPMN orchestriert Aufgaben und Ereignisse. DMN/FEEL trifft fachliche Entscheidungen und wertet Bedingungen aus. Prozesse realisieren Capabilities, aber sind nicht mit ihnen identisch; eine Capability kann auch manuell oder durch ein externes System realisiert werden.

Ebene 4 – Technical Capabilities. Dies ist die **vorgeschlagene Erweiterung**: strukturierte Aussagen wie `identity.account.create` oder `contact.person.create`. Ein semantischer Vertrag bildet Operation, Profil, Vorbedingungen, Eingaben, Ergebnisse, Fehler, Idempotenz und mögliche Side Effects ab.

Ebene 5 – Worker Types und konfigurierte Worker. Ein Worker Type implementiert eine Menge technischer Operationen. Ein konfigurierter Worker ergänzt Zielsystem, Konfiguration und Identitätsmodus. Worker Instances konsumieren durable Jobs. Verschiedene Worker Types dürfen dieselbe Capability nur dann deklarieren, wenn ihre Verträge tatsächlich kompatibel sind.

Ebene 6 – Identity, Policy, Approval. Eine Policy entscheidet anhand serverseitig ermittelter Tatsachen, ob eine konkrete Aktion erlaubt ist. Bei delegierter Ausführung handelt ein Worker nach erteilter Freigabe unter einer externen, mit dem atlas-Principal verknüpften Identität. Genehmigungen müssen an die **konkrete Aktion** gebunden sein, nicht an eine ungeschützte Prozessvariable.

Ebene 7 – Beobachtung, Audit und Wirkung. Ausführen, Messen und Erklären bleiben verschiedene Tätigkeiten. Autorisierungsentscheidungen, Job-Lifecycle und BPMN-Ereignisse liefern Belege; Coverage, Messungen und Graph-Projektionen machen Abhängigkeiten und Wirkungen sichtbar.

Architekturskizze:

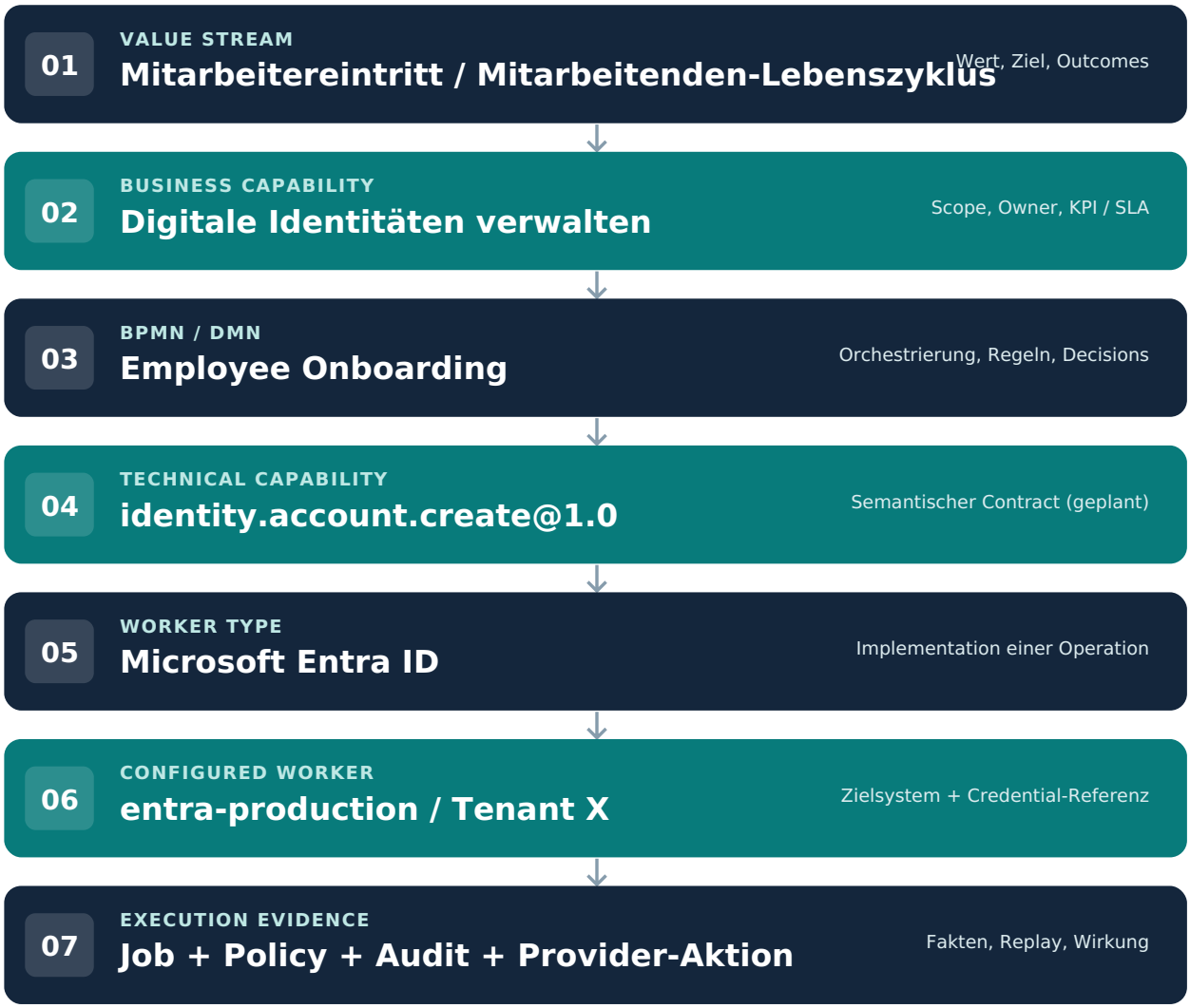


Abbildung 1: Referenzmodell der Capability-driven Automation (Zielbild; kein aktueller Runtime-Graph).

Dies ist **kein** hierarchisches Datenmodell. Der Wertstrom besitzt geordnete Stufen; Business Capabilities bleiben flach; technische Beziehungen sind Kanten eines Graphen, und eine einzelne Capability kann mehrere Realisierungen haben.

4. Der Technical Capability Contract

4.1 Ziel und Abgrenzung

Ein Technical Capability Contract ist ein **Design-Time-Artefakt**, das an bereits existierende Worker-Type-Operationen gebunden wird. Es ist weder ein zweites Business-Capability-Register noch ein neuer Job-Typ oder ein zusätzlicher Ausführungsdienst.

Der aktuelle Code besitzt bereits ein versioniertes Worker-Type-Manifest in `workertype/manifest.go`. Dort umfasst `Spec` unter anderem `JobTypes` und `Operations`, während `Operation` derzeit `ID` und `JobType` enthält. Das ist ein geeigneter Erweiterungspunkt. Auch `GET /api/v1/worker-types` ist bereits verfügbar, liefert nach aktuellem Stand aber eine Worker-Type-Projektion und keine vollständigen semantischen Capability-Verträge.

Vorgeschlagene Vertragsfelder:

Feld	Bedeutung
`id`	Stabiler, herstellerunabhängiger Capability-Identifier (Namensraum).
`version`	Semantische Vertragsversion, getrennt von der Worker-Type-Version.
`operation`	Konkrete, bereits bestehende Operation im Worker Type.
`profile`	Semantisches Profil, z.B. anmeldbares Konto vs. generischer Verzeichniseintrag.
`inputSchemaRef` / `outputSchemaRef`	Typisierte Ein-/Ausgaben und ihre Validierungsregeln.
`preconditions` / `postconditions`	Nachprüfbare Voraussetzungen und zugesicherte Resultate.
`errors`	Normierte Fehlerkategorien sowie Provider-spezifische Fehlerdetails.
`repeatability`	Verhalten bei Retry, Idempotenz-Key, Upsert-Verhalten und Duplikatrisiko.
`consistency`	Sichtbarkeit nach Write, Eventual Consistency, Paging- und Suchgrenzen.
`security`	Erforderlicher Identity-Modus und Ressourcen-/Scope-Voraussetzungen.
`effects`	Klassifikation: read-only, mutating, destructive, externally visible.
`conformance`	Nachweis, mit welchen Contract-Tests die Implementierung geprüft wurde.

Die konkreten Feldnamen sind ein **Entwurf für einen ADR**, keine heute verfügbare API.

4.2 Beispielvertrag (illustrativ, nicht implementiert)

```
apiVersion: atlas.io/v1alpha1
kind: WorkerType
metadata:
  id: atlas.entra
  version: 1.0.0
spec:
  # ... existing fields omitted ...
  technicalCapabilities:            # PROPOSED
    - id: identity.account.create
      version: "1.0"
      profile: interactive-directory-account
      operation: create-user
      inputSchemaRef: "#/schemas/IdentityAccountCreate"
      outputSchemaRef: "#/schemas/IdentityAccountResult"
      repeatability:
        guarantee: non-idempotent-without-external-key
      consistency:
        readAfterWrite: provider-dependent
      effects:
        - mutating
        - identity-provisioning
      security:
        modes: [service]
      conformance: unverified
```

Im Beispiel wäre `technicalCapabilities` eine zusätzliche, zu versionierende Deklaration. Das bestehende Manifest akzeptiert derzeit unbekannte Felder **nicht**; die Erweiterung benötigt deshalb eine explizite Schema-/Parser-Änderung samt Versionierungsentscheidung. Für integrierte Worker muss der Katalog die semantischen Angaben idealerweise aus der **bereits vorhandenen, compilerseitig gemeinsam genutzten Operationstabelle** ableiten, statt einen konkurrierenden zweiten Ausführungskatalog zu schaffen.

4.3 Keine vorschnelle Gleichsetzung von Entra, LDAP und Google Contacts

Domäne / Operation	Entra ID	LDAP	Google Contacts
`identity.account.create`	Kandidat: anmeldbarer Benutzer	**Nur** mit konkretem Schema und Account-Semantik; generisches LDAP-`add` genügt nicht	Nein: Kontakt ist kein Login-Account
`directory.entry.search`	Nur mit definiertem Suchprofil	Kandidat: LDAP-Suche	`search-directory` hat andere Such-/Ergebnissemantik
`contact.person.create`	Kein automatischer Match	Abhängig vom Objektmodell	Kandidat: `create-contact`
`directory.group.membership.add`	Kandidat: Gruppenmitgliedschaft	Mit passendem Schema/Member-Attribut	Gruppen/Labels sind keine Sicherheitsgruppen

Regel: Eine Operation darf eine Technical Capability nur dann als *konform* deklarieren, wenn Contracts und Conformance-Tests die verlangte Semantik belegen. Andernfalls lautet der Status `candidate`, `partial` oder `incompatible` – niemals eine stillschweigende Gleichsetzung.

5. Typisierung, Kompatibilität und Versionierung

Die **Worker-Type-Version** (`atlas.entra@1.0.0`) beschreibt die konkrete Implementierung. Die **Capability-Version** (`identity.account.create@1.0`) beschreibt einen davon unabhängigen, semantischen Vertrag. Beide müssen getrennt bleiben: Ein Worker-Release kann Fehler korrigieren, ohne die öffentliche Capability-Semantik zu ändern; umgekehrt erfordert eine geänderte Semantik möglicherweise eine neue Vertragsversion.

Für den ersten Slice empfehle ich:

- Stabile Namen mit kontrollierten Namensräumen und dokumentierter Terminologie.
- Schema-Kompatibilität strikt prüfen (erforderliche Inputs, Output-Typen, Null-/Fehler-Semantik).
- Profile als zusätzliche Eingrenzung statt einer universellen `Person`-Abstraktion.
- Explizite Konformitätsgrade: `verified`, `declared`, `partial`, `incompatible`.
- Versionspinning beziehungsweise explizite Migrationswarnungen vor einer späteren Runtime-Austauschbarkeit.

Conformance-Beispiele: `create` gefolgt von `get`; Duplikat-Aufruf; Retry nach simuliertem Timeout; Zugriff ohne Scope; ungültige Inputs; Mapping von Provider-Fehlern; Wiederholung nach Reconnect; Prüfung der Suche unmittelbar nach Erzeugung. Nicht jede Eigenschaft lässt sich im Mock belegen. Die Google-Contacts-ADR dokumentiert ausdrücklich noch offene Live-API-Annahmen: Mock-Tests allein dürfen daher keinen vollständigen Konformitätsnachweis vortäuschen.

6. Registrierung, Graph und Panorama

6.1 Ein Graphmodell statt duplizierter Daten

ADR-0420 schlägt einen **deklarativen Katalog von Knotentypen, Identitäten, Datenquellen und Kanten** vor. Der Graph ist dabei eine lesbare Projektion aus den jeweils zuständigen Stores und expliziten Beziehungen; er ist **kein neuer zentraler State Store**. Dies ist die richtige Grundlage für Technical Capabilities.

Neue deklarative Knoten und Kanten könnten sein:

```
[value-stream-stage] --uses--> [business-capability]
[business-capability] --realized-by--> [process-definition]
[process-definition] --requires--> [technical-capability@version]
[worker-type@version] --implements--> [technical-capability@version]
[configured-worker] --instance-of--> [worker-type@version]
[process-deployment] --uses--> [configured-worker]
[policy] --governs--> [worker / action / resource]
```

Diese Beziehungen stammen aus unterschiedlichen Quellen: **authored** (deklarierte Architektur), **derived** (aus Deployment/Katalog abgeleitete Struktur) oder **observed** (tatsächlicher Vorgang). In einer Abfrage muss die Herkunft sichtbar sein. Ein gezeichneter Pfeil beweist weder, dass ein Provider verfügbar ist, noch, dass eine Aktion autorisiert ausgeführt wurde.

6.2 Identität und Abfragegrenzen

Knoten brauchen stabile IDs. Ein Capability-Key ist schon heute ein portabler Geschäftsschlüssel; ein Worker Type besitzt eine globale ID und Version; der konfigurierte Worker ist eine installationsspezifische Bindung; eine konkrete Prozessversion benötigt eine eindeutige Deployment-Identität. Die Graphprojektion muss diese Begriffe auseinanderhalten und Referenzen zur Laufzeit auflösen.

Eine Impact-Abfrage muss ein **Autorisierungsfilter je Knoten und Kante** anwenden. Eingeschränkte Objekte dürfen nicht allein durch Graph-Traversal sichtbar werden. Falls Ergebnisse aus Datenschutz-/Berechtigungsgründen unvollständig sind, muss die Antwort diesen Filterzustand ausweisen. Grössere Traversals laufen ausserhalb des Single-Writer-Hot-Paths und unter Ressourcenbudget.

6.3 Konkrete Panorama-Ansichten

1. **Capability Coverage:** Welche BPMN-Prozesse, Worker und manuellen Leistungen realisieren die Business Capability?
2. **Technical Contract Explorer:** Welche Worker Types implementieren eine Operation in welcher Vertragsversion und mit welchem Konformitätsgrad?
3. **Impact-Analyse:** Welche Value-Stream-Stufen hängen transitiv von einem ausgefallenen Worker oder einer veralteten Capability-Version ab?
4. **Governance View:** Wo bestehen fehlende Policies, nicht getestete Capability-Mappings oder veraltete Bestätigungen?
5. **Evidence Drill-down:** Von einer Geschäfts-Capability in den Prozess, die Job-Ausführung und die zugehörige Autorisierungsentscheidung navigieren – ohne einen laufenden Prozess in das Architekturregister zu kopieren.

7. Identität und Policy-basierte Autorisierung

ADR-0475 verfolgt bereits die Idee der **identity-aware Worker execution**. Im aktuellen Repository ist dies **teilweise implementiert**; eine dokumentierte SharePoint-Beispielstrecke existiert. Das darf nicht als generelle Absicherung aller Worker Types vermarktet werden.

Der vorgesehene Entscheidungsrahmen trennt:

- **Principal / Subject:** Für wen wird die Aktion ausgeführt?
- **Actor / Worker:** Welcher konfigurierte Worker und welche effektive Identität handeln?
- **Action:** Welche konkrete Operation mit welchem Parameter-Digest?
- **Resource:** Welches geschützte Zielobjekt ist betroffen?
- **Context:** Prozessinstanz, Job, Zeitpunkt, Genehmigungen, relevante Policies.

Es gilt **Default Deny** bei fehlenden, ungültigen oder nicht anwendbaren Policies, soweit ein Worker als *governed* konfiguriert ist. Die technische Berechtigung im Zielsystem bleibt die äussere Grenze: atlas kann weder einer nicht vertrauenswürdigen Worker-Implementierung noch einem bereits ausgehändigten langlebigen Service-Credential zuverlässig die Fähigkeit zum Missbrauch entziehen.

Beispiel: Eine Person beantragt die Löschung eines SharePoint-Dokuments. Der Prozess erfasst die Aktion. Die Policy verlangt eine Genehmigung durch eine andere Person. Diese bezieht sich auf den **Digest der konkreten Löschaktion**. Erst nach positiver, dauerhaft protokollierter Entscheidung erhält ein überwacht gestarteter Worker ein passendes, kurzlebiges Token und führt die Operation aus. Eine Prozessvariable `approved=true` ersetzt diese Genehmigung ausdrücklich nicht.

Wichtige Grenzen: Der Token-Broker ist im dokumentierten Stand auf **atlas-supervised Worker Instances** beschränkt. Service-Identitäten sind schwächer erzwingbar als pro Job freigegebene Delegationen. Google-, LDAP- und Entra-Worker sind nicht automatisch durch dieselbe Enforcement-Strecke geschützt. Jede Capability-Mapping- und Routing-Entscheidung muss diese Grenzen offenlegen.

8. Runtime-Ausführung: vom expliziten Worker zur Capability-Anforderung

8.1 Phase A – heutiges Modell beibehalten

Im **ersten Umsetzungsschritt** bleibt ein BPMN-Service-Task einem konkreten Worker und einer bekannten Operation zugeordnet. Der Technical Capability Contract wird lediglich im Katalog, im Compiler für statische Validierung und in den Architekturansichten ausgewertet. Damit entsteht unmittelbar ein Mehrwert ohne Eingriff in Laufzeit-Routing und Recovery.

8.2 Phase B – optionaler Capability Selector (neuer ADR erforderlich)

Eine spätere BPMN-Erweiterung könnte eine abstrakte Anforderung deklarieren:

```
technicalCapability: identity.account.create
capabilityVersion: "1.0"
profile: interactive-directory-account
targetPolicy: corporate-identity
selection: pinned-at-deploy  # future proposed option
```

Der Compiler prüft Vertrag und verfügbare Bindung. Für **`pinned-at-deploy`** wird die konkrete, versionierte Worker-Bindung beim Deployment festgelegt: deterministisch, sichtbar und ohne dynamische Suche im Hot Path. Änderungen benötigen ein neues Deployment beziehungsweise eine explizite Rebind-Operation.

Ein späteres **`resolved-per-job`** wäre wesentlich anspruchsvoller: Zielauswahl und Autorisierung hängen möglicherweise vom aktuellen Zustand ab. Die ausgewählte Worker-ID, Vertragsversion, Zielidentität sowie die für das Replay erforderlichen Routing-Fakten müssen vor externer Sichtbarkeit dauerhaft feststehen. Ein Policy-Neuentscheid beim Replay ist unzulässig. Diese Betriebsart benötigt eine eigene Sicherheits-, Semantik- und Recovery-Analyse, bevor sie als Feature angeboten wird.

8.3 Sequenz einer kontrollierten Ausführung

```
BPMN Task -> Processor -> JobCreated event -> WAL / fsync
          -> state commit -> job activation / lease
Worker Instance -> /jobs/{key}/authorize
          -> serverseitige Policyentscheidung
          -> AuthorizationDecided event -> WAL / fsync
          -> ggf. kurzlebiges Credential fuer zugelassenen Worker
          -> Provider-API-Aufruf
          -> JobCompleted / JobFailed command
          -> Event -> WAL / fsync -> State
```

Der Worker darf das Zielsystem **erst nach der dokumentierten Autorisierungsfreigabe** aufrufen. Bei Leasetimeout, Crash oder Wiederholung können Provider-Aufrufe dennoch erneut erfolgen: das System hat *at-least-once*-Semantik, nicht automatisch *exactly-once*-Side-Effects. Dazu braucht der Capability-Vertrag explizite Informationen über Duplikatvermeidung und Idempotenz.

9. Event-Sourcing, Recovery und Datenminimierung

Eine neue Fähigkeit darf die sechs Invarianten nicht abschwächen:

Invariante	Umsetzung für Capability-driven Automation
I1 – keine Allokationen im Hot Path	Manifest-Parsing, Kompatibilitätsprüfung, Resolver-Aufbau und Semantikprüfung zur Design-/Deployment-Zeit; keine per-Command-Graphabfragen.
I2 – durable before visible	Job, Autorisierungsentscheidung und relevante Routing-Fakten werden dauerhaft, bevor Worker-Aktionen oder Credential-Ausgabe extern sichtbar sind.
I3 – Single Writer	Kein direkter Cross-Partition-Zustandszugriff; persistierte Entscheidungen laufen über den zuständigen Partition-Processor.
I4 – ein `applyToState`	Live- und Recovery-Pfad verwenden dieselbe deterministische Mutation. Keine Policy-Auswertung im Replay.
I5 – compile, don't interpret	Verträge und BPMN-Referenzen werden beim Deployment validiert, indiziert und möglichst fest gebunden.
I6 – Events sind Fakten	Effektive Entscheidung, Version, IDs, Zeit und erforderliche Routing-Fakten werden als Fakten gespeichert; nicht aus mutablem Ist-Zustand rekonstruiert.

Datenminimierung: Weder Credentials noch vollständige Dokumente oder grosse personenbezogene Payloads gehören allein wegen der Capability-Klassifikation ins WAL. Für Governance sind vorrangig stabile Referenzen, Klassifikation, Vertrag/Version, Digest, Entscheidung, Prozess-/Job-Key und fachlich notwendige Outcome-Metadaten relevant. Ein Digest schützt die Genehmigungsbindung, macht aber sensible Daten nicht automatisch ungefährlich: Quellwerte, Korrelationen und Lösch-/Aufbewahrungspflichten brauchen ein eigenes Datenschutzkonzept.

Recovery-Test für jede Runtime-Änderung: Prozess bis zur Entscheidung/Job-Aktivierung ausführen, WAL persistieren, Neustart simulieren, Events replayen, Live- mit Replay-State vergleichen, dann die zulässigen Worker-Retry- und Duplicate-Szenarien prüfen. Ein Lauf, der den Provider schon angesprochen hat, kann nach einem Crash erneut angesprochen werden; das muss die Contract-Semantik offen kommunizieren.

10. APIs und Datenmodell – technischer Schnittstellenentwurf

Bestehende API/Oberflächen: `GET /api/v1/worker-types`, Capability-Register unter `/api/v1/capabilities`, Panorama sowie Worker-/Job-APIs. Policy- und Authorize-Endpunkte bestehen im Umfang von ADR-0475. Diese bestehenden Verträge sollen nicht unbemerkt umgedeutet werden.

Vorgeschlagene additive Reads (noch keine verfügbaren Endpunkte):

```
GET /api/v1/technical-capabilities
GET /api/v1/technical-capabilities/{id}/versions/{version}
GET /api/v1/worker-types/{id}/capabilities
GET /api/v1/technical-capabilities/{id}/implementations
```

Die Entscheidung, ob ein separater Read-Endpunkt sinnvoller ist als eine Erweiterung von `GET /api/v1/worker-types`, gehört in den ADR. Wichtig sind stabile Pagination/Filterung, versionierte Responses, Sichtbarkeitsregeln, keine Ausgabe von Secrets und ein einziger autoritativer Katalog.

Illustrativer Response:

```
{
  "id": "identity.account.create",
  "version": "1.0",
  "profile": "interactive-directory-account",
  "implementations": [
    {
      "workerTypeId": "atlas.entra",
      "operation": "create-user",
      "contractStatus": "declared",
      "testedAgainstProvider": false
    }
  ]
}
```

Dieser Response ist ein **Zieldesign**, nicht der tatsächliche heutige API-Output.

11. Governance, Bedrohungen und betriebliche Grenzen

Risiko	Architekturmassnahme
Scheinbare Provider-Austauschbarkeit	Strenge Profile, Conformance-Tests, explizite fehlende Guarantees und Versionsbindung.
Unbefugte Ausführung	Default-Deny-Policy, serverseitige Fakten, bindende Genehmigungen, Worker-Identity- und Token-Grenzen.
Compromised Worker / Service Account	Kurzlebige per-job Tokens wo erzwingbar, Least Privilege im Zielsystem, Isolation; keine Behauptung vollständiger Kontrolle.
Doppelte externe Side Effects	Idempotenz-Key, Vorher-/Nachher-Prüfung, deduplizierende Provider-Operation oder sichtbarer Incident.
Graph als Datenleck	Per-Node-Authorisierung, eingeschränkte Beziehungen maskieren, Filterstatus ausweisen.
Vertrag driftet von Implementierung	Gemeinsame Operationstabelle, CI-Konformität, Nachweise und Überprüfungsdatum.
Neue Last auf Processor	Ableitungen ausserhalb des Loops, Snapshot-Reads und Kostenbudgets.
Veraltete Business Architecture	Bestätigungsdatum des Capability-Registers, Gap Report, klare Provenienz.
Versteckte Credential-Leaks	Secret-Referenzen statt Werte, kein Logging von Tokens, auditierbare Rotation.

Die Fähigkeit, Workflows im Namen einer Person auszuführen, ist kein alleinstehendes Alleinstellungsmerkmal. Das differenzierende Ziel ist die **nachvollziehbare Kombination** aus Business-Architektur, semantischen Verträgen, Policies, Entscheidungen, Recovery und Wirkungsanalyse – unter klar ausgewiesenen Garantien.

12. Roadmap: realistische Umsetzung in vertikalen Slices

Zuordnung: Erweiterung von Milestone B (Business Architecture) für Beziehungen und Katalogmetadaten; Abhängigkeit von der vorgeschlagenen Graph-Architektur ADR-0420; autorisierte Runtime-Ausführung unter ADR-0475. Die bestehenden Decisions setzen bewusst Grenzen: technische Integration Capabilities sollen **keine separaten Business-Capability-Records** werden.

Slice	Liefergegenstand	Erfolgskriterium / Abnahme
T0 – Architekturentscheidung	ADR „Technical Capability Contract“, Schema, Identität, Kompatibilität, Status und Zuordnung zu ADR-0208/0305/0420/0475	Keine Invariantenverletzung, eindeutige Eigentümerschaft aller Daten und API-Kompatibilität.
T1 – Katalogvertikale	Manifest-/Katalog-Erweiterung, erste Entra-/LDAP-/Google-Contacts-Mappings, read-only API	Contract-Tests grün; Compiler/Worker verwenden weiterhin dieselbe Operationstabelle. Keine Routing-Änderung.

Slice	Liefergegenstand	Erfolgskriterium / Abnahme
T2 – Modellvalidierung	BPMN-Modell zeigt „required/provided capability“ und Warnungen bei semantischer Unvereinbarkeit	Ungültige Mappings werden vor Deployment erklärt; bestehende BPMN-Modelle funktionieren unverändert.
T3 – Graph und Panorama	Knoten/Kanten in ADR-0420-Modell, serverseitige Impact-Abfragen, Anzeige der Herkunft	Nachvollziehbare Abfrage von Value-Stream-Stufe zu Worker; Per-Node-Authorisierung geprüft.
T4 – Identity/Policy-Integration	Capability-Aktion in Policy- und Governance-Ansichten; bestehende SharePoint-Strecke als Referenz	Aktion und Policy-Entscheid nachvollziehbar, keine Verallgemeinerung auf ungeprüfte Worker.
T5 – Optionales Runtime-Routing	Gesonderter ADR, zuerst `pinned-at-deploy`, später ggf. `resolved-per-job`	Recovery-/Race-/Fencing-/Idempotenztests; Provenienz jeder Auswahl; keine Hot-Path-Allokationen.

TDD und Definition of Done: zuerst fehlschlagende Tests, minimale Implementation, Refactoring; bei Änderungen an Job/Persistenz/Prozessverhalten mit Recovery-Vergleich. Vor Abschluss: `go build ./...`, `go test ./...`, `go test -race -timeout=45m ./...`, `go vet ./...`, `gofmt -l .` ohne Ausgabe. Zusätzlich passende E2E-Tests sowie gegebenenfalls Windows-CI.

13. Empfohlener Proof of Concept

Der erste PoC sollte absichtlich **keine** automatische Provider-Auswahl vornehmen. So wird die fachliche Idee bewiesen, ohne zu früh die schwierigste Runtime-Frage zu lösen.

Szenario: Mitarbeiter-Onboarding mit zwei vorhandenen Workern und einer Capability-Ansicht in Panorama.

1. Ein Value Stream enthält die Stage *Digitale Identität bereitstellen*.
2. Eine Business Capability *Identity Lifecycle Management* ist im Register erfasst und mit dem Onboarding-Prozess realisiert.
3. Der BPMN-Prozess enthält einen expliziten `entra.create-user`-Task. Der Worker-Katalog zeigt dazu den **deklarierten** Technical Capability Contract `identity.account.create@1.0`.
4. Eine zweite LDAP-Operation wird als `candidate` oder `incompatible` angezeigt, bis Schema und tatsächliche Account-Semantik nachweislich passen.
5. Die technische Graph-Sicht stellt die Kette Value Stream → Capability → BPMN → Operation → Worker dar.
6. Wird der Worker deaktiviert, zeigt die Impact-Abfrage die **potenziell** betroffenen fachlichen Stufen. Die Anzeige unterscheidet eine strukturelle Abhängigkeit von einem tatsächlich eingetretenen Prozessfehler.

Messbare PoC-Ziele: korrekte technische Verträge ohne falsche Gleichsetzungen; reproduzierbare Katalogauskunft; erfolgreiche Graph-Abfrage ohne Datenkopie; keine Regression in BPMN-/Job-/Recovery-Tests; verständliche Impact-Ansicht; klar dokumentierte Grenzen der Policy-Durchsetzung.

14. Offene Entscheidungen vor der Implementierung

1. **Normierung:** Soll der Capability-Vertrag im bestehenden `workertype.Manifest.Spec` liegen, über ein ergänzendes Manifest-Element referenziert werden oder in den jeweiligen Operationstabellen definiert und dort kompiliert werden?
2. **Vertragsversion:** Reicht SemVer mit Profilen und Schemas oder brauchen wir eine explizite Major-Kompatibilitätsregel für die semantische Operation?
3. **BPMN-Repräsentation:** Zusatz-Metadaten auf bestehendem Service Task oder ein neues, optionales authoring-fähiges Binding? Keine nicht-BPMN-konforme Execution-Logik in der UI.
4. **Auswahlmodus:** Bleibt Capability-based Binding zunächst reine Design-Time-Prüfung, wie empfohlen? Bei Runtime-Auswahl müssen die neuen Regeln als ADR entschieden werden.
5. **Conformance:** Was gilt als durch echte Provider-Tests bestätigt, was bleibt ein Mock-Nachweis, und wie wird die Test-/Versionsmatrix gepflegt?
6. **Graph-Priorität:** Welcher kleinste Graph-Slice liefert Impact-Abfragen ohne den gesamten ADR-0420-Vollausbau vorauszusetzen?
7. **Policy-Enforcement:** Für welche Worker Types können wir mehr als kooperatives Service-Credential-Enforcement nachweisen?

Diese Punkte sind **Entscheidungsbedarf**, keine versteckten Vorbedingungen, um mit T0/T1 zu starten.

15. Fazit und technische Vision

Die strategische Vision ist grösser als ein „Universal Directory Worker“. atlas kann künftig **die Beziehung zwischen dem Wert einer Organisation und der tatsächlich verantworteten Automation formal erklären und überprüfen**. Dafür braucht es keine zweite Prozess-Engine, keinen separaten Business-Capability-Datenbestand und keinen eigenständigen Graphdatenbank-Zwang. Es braucht präzise technische Verträge, stabile Identitäten und Kanten, nachvollziehbare Provenienz, überprüfbare Policies sowie dieselben dauerhaften Fakten, auf denen atlas bereits heute beruht.

Der erste konkrete Schritt ist klein und wertvoll: **Ein Technical Capability Contract im Worker-Type-Katalog, mit Entra, LDAP und Google Contacts als bewusst kontrastierenden Referenzfällen.** Darauf folgen Graph und Panorama. Erst zuletzt – und nur nach eigener ADR – folgt die flexible Runtime-Auswahl.

> **Die Vision:** Nicht nur automatisieren, *was* technisch möglich ist. Sondern beweisen, *welchen Geschäftswert* eine Automation ermöglicht, *welcher Vertrag* dafür gilt, *wer* sie ausführen durfte, *wie* sie ausgeführt wurde und *was* tatsächlich geschehen ist.

Referenzen und Ist-Stand (Repository geprüft am 24.10.2026)

Quelle	Bedeutung / Status
README.md, AGENTS.md, ARCHITECTURE.md	Produkt- und Engine-Architektur; Invarianten und Projektregeln.
ROADMAP.md, Milestone B	Business Architecture B0–B8/B10 weitgehend umgesetzt; B9 noch offen.
ADR-0305, api/capability/model.go	Business Capability-/Value-Stream-Records: **Accepted / Partial**.
ADR-0308	Panorama-Register-Bindings: **Accepted / Landed**.
ADR-0203	Worker Type / Worker / Worker Instance: **Accepted / Landed**.
ADR-0208, workertype/manifest.go, api/workertypes.go	Worker-Type-Manifest und Katalogprojektion: **Partial**.
ADR-0476, Google-Contacts-Katalog	Google Contacts: **Landed**, reale Provider-Annahmen noch zu prüfen.
ADR-0420	Einheitliches Graphmodell: **Proposed / Not started**.
ADR-0475, authz/request.go	Identity-aware Worker Execution: **Proposed / Partial**.
Invarianten	I1–I6 verbindlich.

Hinweis zur Bewertung: Aussagen zum aktuellen Code beziehen sich auf die genannten Repository-Dokumente und geprüften Dateien, nicht auf eine vollständige Auditierung aller Packages oder eine erneute Ausführung der Tests. Alle als *proposed*, *illustrative* oder *future* gekennzeichneten Strukturen sind noch keine öffentliche atlas-Schnittstelle.